

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ОРЛОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ И.С. ТУРГЕНЕВА»

Кафедра информационных систем и цифровых технологий

Работа допущена к защите

« 2 » 11 20 25 г. Руководитель

КУРСОВАЯ РАБОТА

по дисциплине «Базы данных»

на тему: «Проектирование и разработка базы данных для ведения личных дел
сотрудников в отделе кадров»

Студент Кор Королев А. С.

Шифр 220893

Институт приборостроения, автоматизации и информационных технологий

Направление подготовки 09.03.03 Прикладная информатика

Направленность (профиль) Интеллектуальная обработка данных

Группа 31ПИ

Руководитель Ры Рыженков Д.В.

Оценка: « 5 » Дата 7.11.2025

Орел 2025

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ОРЛОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ И.С. ТУРГЕНЕВА»

Кафедра информационных систем и цифровых технологий

УТВЕРЖДАЮ:

 и.о. зав. кафедрой

«14» 02 2025 г.

ЗАДАНИЕ
на курсовую работу

по дисциплине «Базы данных»

Студент Королев А. С.

Шифр 220893

Институт приборостроения, автоматизации и информационных технологий

Направление подготовки 09.03.03 Прикладная информатика

Направленность (профиль) Интеллектуальная обработка данных

Группа 31ПИ

1 Тема курсовой работы

«Проектирование и разработка базы данных для ведения личных сотрудников
в отделе кадров»

2 Срок сдачи студентом законченной работы «20» мая 2025

3 Исходные данные

Данные личных дел сотрудников в отделе кадров.

4 Содержание курсовой работы

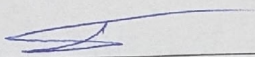
Описание предметной области: ведение личных дел сотрудников в отделе кадров;

Проектирование схемы базы данных;

Реализация базы данных.

5 Отчетный материал курсовой работы

Пояснительная записка курсовой работы; SQL скрипты для создания и работы с базой данных.

Руководитель  Рыженков Д.В.

Задание принял к исполнению: «14» февраля 2025

Подпись студента 

Содержание

Введение	5
1 Описание предметной области	6
2 Разработка схем базы данных	8
2.1 Разработка концептуальной схемы	8
2.2 Разработка логической схемы базы данных	10
2.3 Разработка физической схемы базы данных	13
3 Реализация базы данных	16
3.1 Скрипты создания базы данных	16
3.2 Скрипты заполнения базы данных	18
4 Реализация запросов к базе данных	19
5 Триггеры и процедуры к базе данных	21
5.1 Разработка триггеров	21
5.2 Разработка процедур	23
Заключение	25
Список использованных источников	26
Приложение А (обязательное)	27
Приложение Б (обязательное)	31
Приложение В (обязательное)	33
Приложение Г (обязательное)	36

Введение

Актуальность темы обусловлена возрастающей потребностью современных организаций в эффективных системах управления персоналом. В условиях цифровой трансформации и увеличения объемов кадровой документации традиционные методы ведения бумажных личных дел становятся недостаточно эффективными, что приводит к затруднениям в оперативном доступе к информации, сложностям в анализе кадровых данных и повышению вероятности ошибок при обработке документов.

Кадровая служба является ключевым структурным подразделением любой организации, отвечающим за формирование, учет и движение персонала. От эффективности ее работы напрямую зависят такие важные аспекты деятельности предприятия, как подбор квалифицированных специалистов, обеспечение непрерывности бизнес-процессов, соблюдение трудового законодательства и формирование кадрового резерва.

Целью данной курсовой работы является разработка реляционной базы данных, которая обеспечит комплексное решение задач кадрового учета. База данных должна охватывать все аспекты работы с персоналом: от первичного приема на работу до ведения полной истории трудовой деятельности сотрудника, включая переводы между должностями и структурными подразделениями, учет отпусков, отслеживание повышения квалификации и другие кадровые мероприятия.

Практическое значение данной работы заключается в возможности внедрить разработанную БД как основу для деятельности кадровых служб предприятий малого и среднего бизнеса, где использование дорогостоящих коммерческих систем управления персоналом может быть экономически нецелесообразным.

1 Описание предметной области

Предметной областью курсовой работы является деятельность отдела кадров, связанная с учетом сотрудников и управлением трудовыми отношениями внутри организации. Отдел кадров выполняет ключевую функцию в любой компании — он отвечает за оформление, хранение и обработку информации о персонале, начиная с момента приёма сотрудника на работу и заканчивая его увольнением.

В центре внимания находятся следующие процессы: оформление трудовых договоров. По итогам собеседования принимается решение взять сотрудника в компанию, с ним заключается трудовой договор, договору присваивается соответствующий номер, фиксируется информация о дате заключения договора, записывается соответствующий статус для договора, статусов для договора бывает несколько, тип договора (бессрочный / срочный). На основе трудового договора формируется приказ о приеме сотрудника на работу, сотрудник занимает определенную должность в определенном отделе. Стоит отметить, что в предметной области существуют приказы (они могут быть 4 типов: прием на работу, перевод сотрудника, оформление отпуска и увольнение), каждый с соответствующим описанием. Также на этих этапах вносится информация о предыдущих местах работы сотрудника (организация, должность, дата начала работы, дата окончания). Когда сотрудник начинает занимать должность, в отделе кадров фиксируют информацию о том, что сотрудник вступил в должность, также сотрудник, как только он вступил в должность, вступает и в определенный отдел, о нем также сохраняется информация, то есть в отделе кадров должна быть информация о том, в каких отделах и на каких должностях работает или работал сотрудник. И, конечно, сохраняются номер трудового договора и номер приказа, по причине которого и происходят данные изменения.

В определенный момент сотрудник может перевестись в другой отдел на другую должность. Данный процесс проходит успешно, если на это имеется

подтвержденный приказ о переводе сотрудника. В таком случае заносится информация о переводе, дата перевода, старый отдел и старая должность сотрудника, и новые отдел и должность, и конечно, заносится номер приказа о переводе. При переводе трудовой договор должен быть переоформлен. Каждому сотруднику соответствует приказ, причем для всех они разные. То есть в приказе перечисляется только один сотрудник. Данный процесс протекает при наличии заключенного трудового договора.

Также сотрудник в праве взять отпуск. В таком случае заносится информация об отпуске сотрудника (номер приказа о назначении отпуска, номер трудового договора сотрудника, тип отпуска (оплачиваемый / неоплачиваемый), и вносится дата начала отпуска и его окончания). Отпуск так же, как и все остальные процессы назначается на основании приказа и наличия заключенного трудового договора.

В компании сотрудники также могут проходить обучающие курсы. Они могут быть от этой же компании или же от сторонней организации. О каждом прохождении заносится информация о номере сотрудника, проходящем обучение, наименование курса, обучающая организация, дата начала и окончания прохождения курса, сертификат о завершении курса и оценка прохождения (удовлетворительно, хорошо, отлично). Данный процесс также основан на приказе и наличии трудового договора сотрудника.

Ну и наконец сотрудник может уволиться из компании. В таком случае заносится соответствующая информация о данном процессе (номер увольнительной, дата увольнения, причина и конечно номер приказа), в трудовом договоре ставится статус «уволен».

Таким образом, предметная область охватывает весь цикл управления персоналом, и цель базы данных — обеспечить эффективный учет, автоматизацию отчетности и поддержку принятия решений в сфере управления трудовыми ресурсами.

2 Разработка схем базы данных

2.1 Разработка концептуальной схемы

Концептуальная диаграмма — это наглядное представление основных сущностей предметной области и связей между ними. Она используется на этапе проектирования базы данных для того, чтобы показать, какие объекты важны в системе, какие данные о них нужно хранить и как эти данные взаимосвязаны.

Основная цель концептуальной диаграммы — помочь разработчикам, аналитикам и заказчикам лучше понять структуру системы, согласовать требования и убедиться, что учтены все важные элементы и взаимосвязи до начала программной реализации.

Проектируемая концептуальная диаграмма отражает ключевые сущности и взаимосвязи базы данных для ведения личных дел сотрудников. В системе учитываются сотрудники, история трудовых книжек, приказы, отпуска, обучающие курсы, история занимаемых должностей и отделы. Для разработки концептуальной схемы был использован онлайн сервис для создания диаграмм Draw.io.

Концептуальная схема в нотации Чена представлена на рисунке 1.

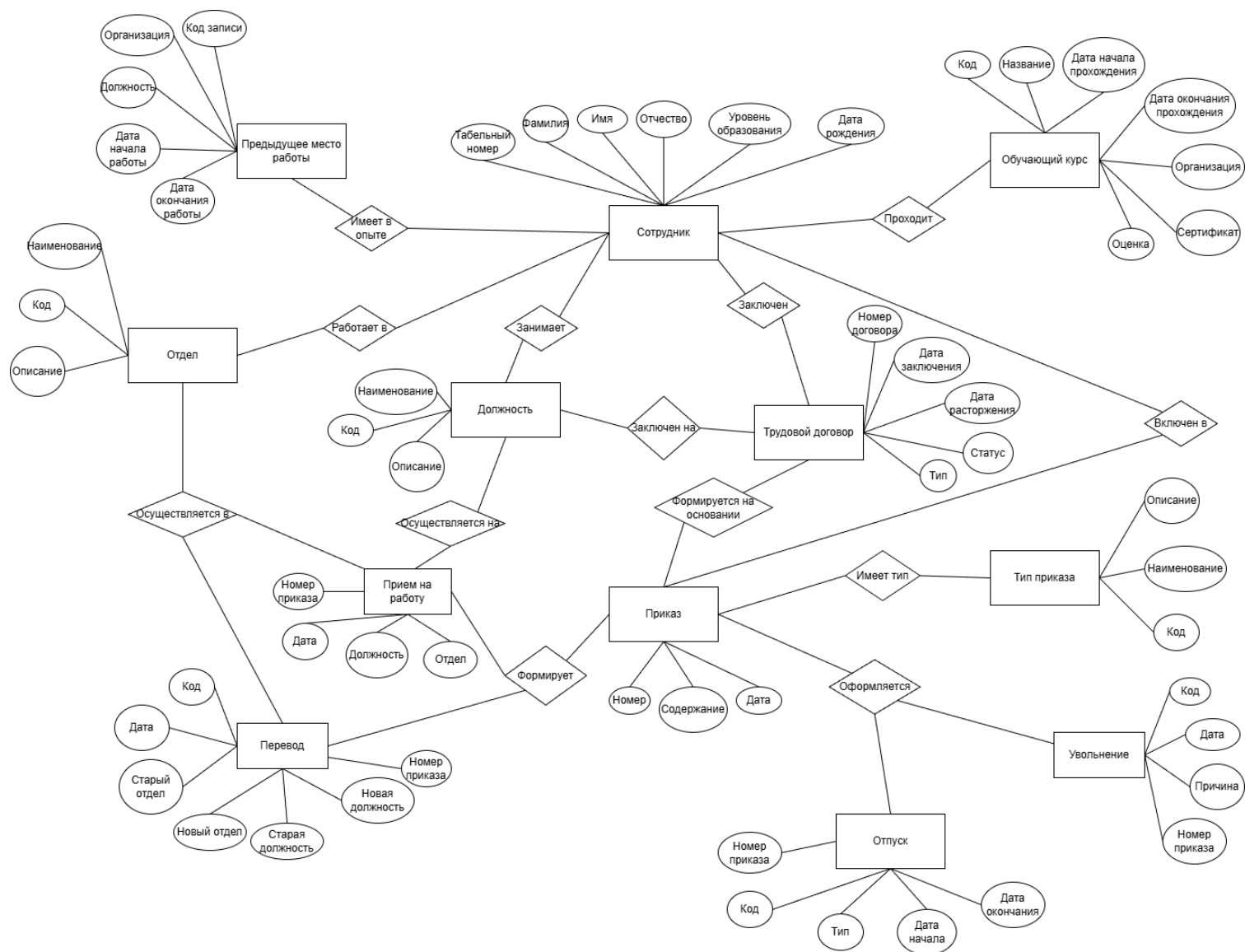


Рисунок 1 - Концептуальная схема в нотации Чена

2.2 Разработка логической схемы базы данных

Логическая модель представляет собой модель базы данных, которая не привязана к конкретной СУБД. В ней выделяют конечные сущности и связи между ними.

Логическое проектирование базируется на концептуальной модели, как правило представленной в виде ER-диаграммы. Для реляционной модели данных существуют формальные правила преобразования концептуальной схемы в логическую — с построением таблиц, определением ключей и связей. Одним из ключевых этапов логического проектирования является нормализация отношений.

Нормализация базы данных позволяет устранить избыточность информации, минимизировать аномалии при вставке, обновлении и удалении записей, а также обеспечить целостность и согласованность данных.

Достаточно привести структуру базы к нормальной форме Бойса–Кодда (BCNF), так как она гарантирует, что все неключевые столбцы зависят исключительно от первичного ключа. Это исключает транзитивные зависимости и повышает логическую строгость структуры.

Для построения логической схемы базы данных использовался онлайн-сервис drawio. Этот инструмент позволяет визуализировать ER-диаграммы, задавать сущности, их атрибуты и связи.

На основании анализа предметной области и концептуальной схемы, при переходе к логической модели в схему были добавлены вспомогательные сущности, которые упрощают структуру данных и повышают её гибкость. К таким сущностям относятся:

- Прохождение курса – позволяет сохранять в базе данных историю о прохождении сотрудником обучающих курсов.
- Статус_выбранного_договора – позволяет сохранять в базе данных информацию о статусе трудового договора.

Добавление этих сущностей делает структуру базы данных более масштабируемой и читаемой. Также, в каждую таблицу добавлены вспомогательные первичные ключи в виде атрибутов id, обеспечивающих уникальность записей и упрощающих реализацию связей между таблицами. Многие связи в логической схеме реализованы по типу «один-ко-многим» и являются неидентифицирующими, то есть дочерние таблицы сохраняют независимость от родительских, а внешние ключи не входят в состав первичных.

Схема базы данных приведена к нормальной форме Бойса–Кодда (BCNF). Все поля в каждой таблице содержат только атомарные значения, что соответствует требованиям первой нормальной формы.

Каждое неключевое поле зависит от всего первичного ключа, а не от его части — это соответствует второй нормальной форме.

Также устранены транзитивные зависимости: ни одно неключевое поле не зависит от другого неключевого поля, что удовлетворяет требованиям третьей нормальной формы.

Все зависимости в таблицах основаны только на ключах — то есть, любые зависимости идут исключительно от суперключей, что и обеспечивает соответствие нормальной форме Бойса–Кодда.

Для разработки логической схемы был использован онлайн сервис для создания диаграмм Draw.io.

Логическая схема проектируемой базы данных представлена на рисунке 2.

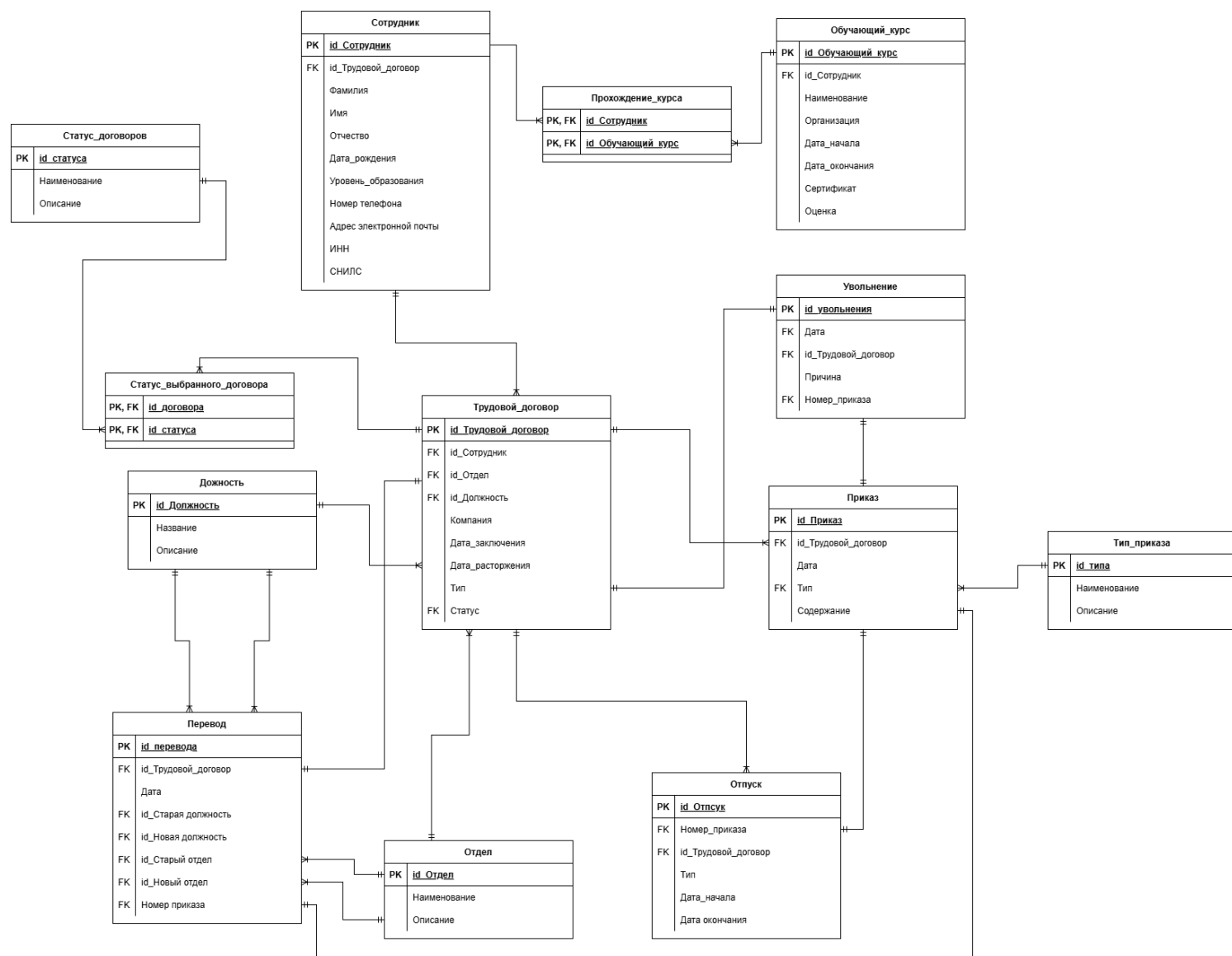


Рисунок 2 – Логическая схема

2.3 Разработка физической схемы базы данных

После создания концептуальной и логической моделей, следующим шагом является построение физической схемы базы данных. Этот этап фокусируется на определении физической структуры хранения данных, которая зависит от возможностей выбранной СУБД.

Физическая схема должна точно соответствовать логической модели, но при этом обеспечивать эффективное хранение и быстрый доступ к данным. Связи между объектами данных реализуются с помощью ключей: уникальный ключ главной таблицы становится ключом связи в связанной таблице. В подчиненной таблице ключ связи может быть частью её уникального ключа или отдельным полем, не входящим в первичный ключ. Такой ключ называется внешним ключом.

Проектируемая база данных использует как первичные, так и внешние ключи. В физической схеме (рисунок 3) показаны как идентифицирующие, так и неидентифицирующие связи между таблицами.

Ниже представлена таблица соответствия обозначений на логической (на русском языке) и на физической (на латинице) ER диаграммах (таблица 1).

Таблица 1 – Соответствие обозначений на логической (на русском языке) и на физической (на латинице) ER диаграммах

Обозначение на логической диаграмме	Обозначение на физической диаграмме
1	2
Сотрудник	employee
Статус_договоров	contract_status
Обучающий курс	training_course
Прохождение_курса	course_completion
Должность	job_position
Статус_выбранного_договора	contract_status_assignment

Продолжение таблицы 1

1	2
Отдел	department
Отпуск	vacation
Приказ	work_order
Тип_приказа	order_type
Сотрудник	employee
Перевод	transfer
Трудовой_договор	employment_contract
Дата_начала	start_date
Дата_окончания	end_date
Тип	type
Статус	status
Создано_в	created_at
Название	title
Описание	description
Оценка	mark
Наименование	name
Компания	company
Организация	organization
Сертификат	certificate
Результат	result
Фамилия	last_name
Имя	first_name
Отчество	middle_name
Дата_рождения	birth_date
Уровень_образования	education

Обновлено_в	updated_at
Продолжительность	days_count
Дата_приказа	order_date

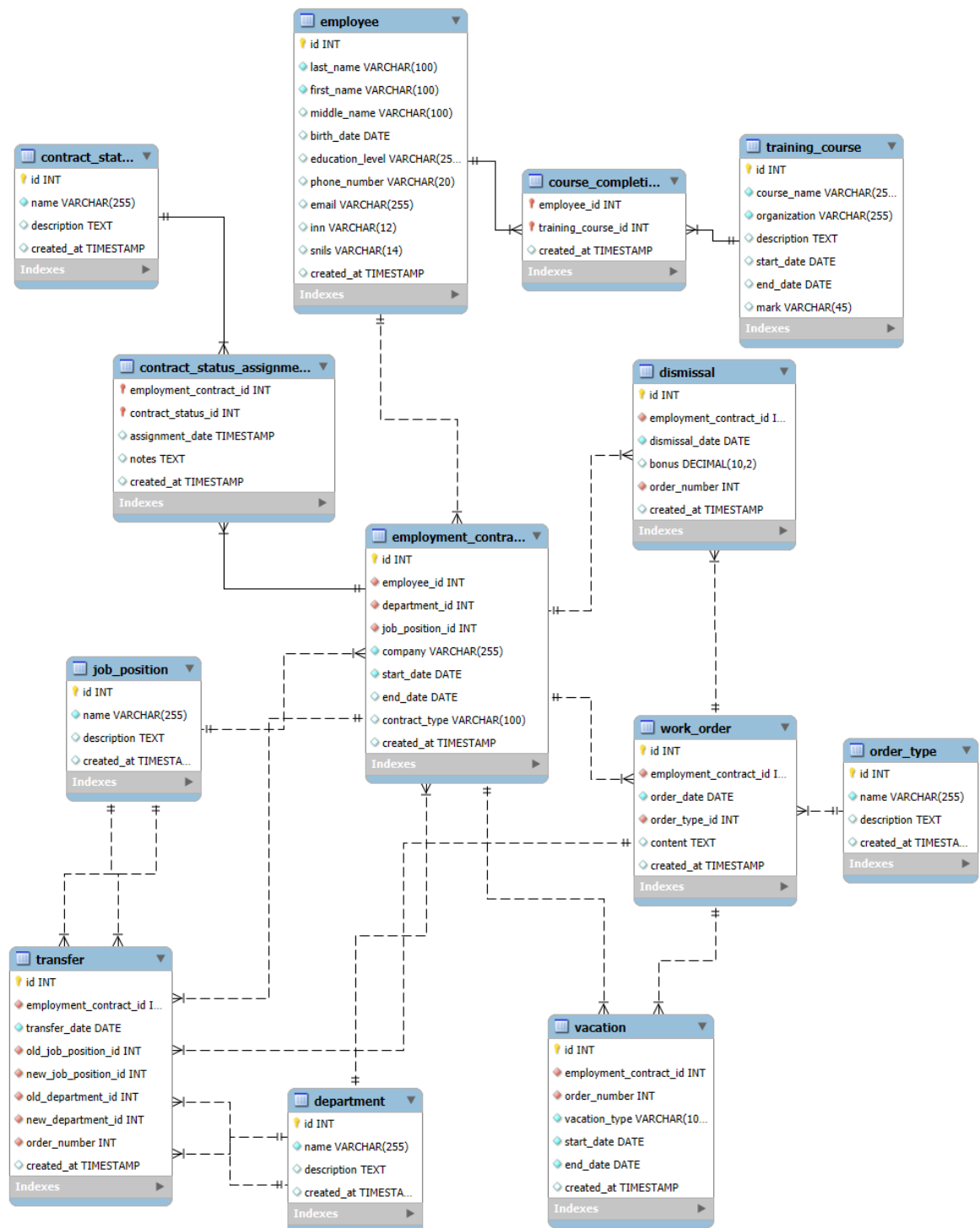


Рисунок 3 - Физическая схема

3 Реализация базы данных

3.1 Скрипты создания базы данных

После построения физической схемы необходимо реализовать базу данных. База данных создается с помощью запросов для создания и заполнения таблиц.

Пример создания таблицы Отдел:

```
CREATE TABLE IF NOT EXISTS Department (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    description TEXT,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Строка **CREATE TABLE** Department - это команда для создания новой таблицы с именем Department.

Строка id **INT** **AUTO_INCREMENT** **PRIMARY KEY**:

id - имя поля (столбца), это уникальный идентификатор каждого отдела.

INT - тип данных: целое число.

AUTO_INCREMENT - автоматическое увеличение значения при добавлении новой строки. Это значит, что при добавлении записи ID будет увеличиваться на 1.

PRIMARY KEY - объявляет этот столбец как первичный ключ. Он обеспечивает уникальность каждой записи и используется для связи с другими таблицами.

Строка name **VARCHAR**(**255**) **NOT NULL**:

name – наименование отдела.

VARCHAR(**255**)- символьный тип данных, максимум 255 символов.

NOT NULL – значения не должны быть NULL.

Строка description **TEXT**:

Description – описание отдела.

TEXT - тип данных для хранения длинных текстовых строк, таких как статьи или комментарии. В отличие от VARCHAR, TEXT не требует указания максимальной длины при создании таблицы и может хранить значительно больший объем данных.

Строка created_at **TIMESTAMP DEFAULT CURRENT_TIMESTAMP:**

created_at – когда была создана запись

TIMESTAMP - тип данных, который используется для хранения даты и времени

DEFAULT CURRENT_TIMESTAMP - это способ установить значение по умолчанию для столбца, которое будет автоматически заполняться текущей датой и временем при вставке новой строки, если для этого столбца не было явно указано другое значение.

Скрипты создания таблиц представлены в приложении А.

3.2 Скрипты заполнения базы данных

Скрипт для заполнения данной таблицы выглядит следующим образом:

пример - заполнение таблицы "Job_position".

```
INSERT INTO Job_position (name, description) VALUES  
( 'Manager', 'Department head'),  
( 'Developer', 'Software developer'),  
( 'Analyst', 'Business analyst'),  
( 'QA Tester', 'Quality assurance specialist');
```

INSERT INTO - команда для добавления новых строк в таблицу.

Job_position - имя таблицы, в которую вставляются данные.

VALUES - ключевое слово, после которого перечисляются значения для вставки.

'Manager' – значение, которое будет вставлено в столбец name (наименование)

'Department head' - значение, которое будет вставлено в столбец description (описание)

Скрипты заполнения разработанной базы данных представлены в приложении Б.

4 Реализация запросов к базе данных

Запросы в языке SQL нужны для взаимодействия с базой данных. Их основная цель — получение, добавление, изменение или удаление информации, хранящейся в таблицах базы данных.

Ниже представлен пример запроса к базе данных, с помощью которого можно определить самые популярные обучающие курсы среди сотрудников:

```
SELECT
    tc.course_name,
    tc.organization,
    tc.duration_days,
    COUNT(cc.employee_id) AS enrolled_employees_count,
    GROUP_CONCAT(CONCAT(e.last_name, ' ', e.first_name)
SEPARATOR ', ') AS enrolled_employees
FROM Training_Course tc
LEFT JOIN Course_Completion cc ON tc.id =
cc.training_course_id
LEFT JOIN Employee e ON cc.employee_id = e.id
GROUP BY tc.id, tc.course_name, tc.organization,
tc.duration_days
ORDER BY enrolled_employees_count DESC;
```

SELECT — команда, с помощью которой извлекаются данные из таблиц.
FROM Training_Course tc — основная таблица, из которой начинается выборка (таблица курсов).

AS – алиас, который приравнивает какое-то значение. В данном случае можно сократить название таблицы.

CONCAT - функция, которая объединяет (конкатенирует) две или более строки в одну.

LEFT JOIN - это оператор, который объединяет две таблицы и возвращает все записи из левой таблицы, а также соответствующие записи из правой таблицы.

ON - условие, по которому происходит соединение.

В данном запросе происходит выборка следующих данных: название курса, обучающая организация, продолжительность прохождения курса и

количество сотрудников на курсе. В этом случае происходит соединение таблиц с сотрудниками и курсами, затем происходит группировка по атрибутам курса, и уже затем идет сортировка курсов по популярности.

Таким образом, данный SQL-запрос предназначен для анализа востребованности обучающих курсов среди сотрудников компании. Он позволяет определить самые популярные курсы, выявить предпочтения сотрудников в обучении и оптимизировать программу корпоративного обучения.

Скрипты запросов к разработанной базе данных представлены в приложении В.

5 Триггеры и процедуры к базе данных

5.1 Разработка триггеров

Триггеры (triggers) — это специальные автоматические действия, которые выполняются при изменении данных в таблицах: при вставке (INSERT), обновлении (UPDATE) или удалении (DELETE).

Вот пример создания триггера для проверки дат отпуска:

```
DELIMITER //
CREATE TRIGGER check_vacation_dates
BEFORE INSERT ON vacations
FOR EACH ROW
BEGIN
    IF NEW.start_date >= NEW.end_date THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Дата начала отпуска должна
быть раньше даты окончания';
    END IF;

    IF NEW.days_count IS NULL THEN
        SET NEW.days_count = DATEDIFF(NEW.end_date,
NEW.start_date) + 1;
    END IF;
END//
DELIMITER ;
```

DELIMITER // - Изменяет разделитель операторов на "//". Это необходимо, потому что внутри триггера есть точка с запятой, и стандартный разделитель ";" прервет создание триггера.

CREATE TRIGGER check_vacation_dates - создает триггер с именем check_vacation_dates.

BEFORE INSERT ON vacations - определяет, что триггер будет срабатывать *перед* каждой операцией вставки новой строки в таблицу vacations.

FOR EACH ROW - указывает, что триггер будет выполняться для каждой вставляемой строки.

BEGIN - начало тела триггера. Здесь находится логика, которая будет выполняться.

```
IF NEW.start_date >= NEW.end_date THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Дата начала отпуска должна
быть раньше даты окончания';
END IF;
```

Здесь сравнивается дата начала отпуска и дата конца отпуска, если начальная дата больше конечной, то выдается ошибка и соответствующее текстовое сообщение.

```
IF NEW.days_count IS NULL THEN
    SET NEW.days_count = DATEDIFF(NEW.end_date,
NEW.start_date) + 1;
END IF;
```

Здесь проверяется продолжительность отпуска, если значение NULL, то задается новое значение, равное разности конечной даты и начальной даты + 1.

END IF; - Конец условного оператора IF.

END - Конец тела триггера.

// - Конец определения триггера (с использованием измененного разделителя).

DELIMITER ; - Возвращает разделитель операторов к стандартному значению ";".

Триггеры к базе данных представлены в приложении Г.

5.2 Разработка процедур

Процедуры (stored procedures) — это заранее сохранённые блоки SQL-кода, которые можно вызывать многократно с разными параметрами.

Вот пример процедуры, которая добавляет нового сотрудника:

```
CREATE PROCEDURE AssignContractStatus(  
    IN p_employment_contract_id INT,  
    IN p_contract_status_id INT,  
    IN p_notes TEXT  
)  
BEGIN  
    INSERT INTO Contract_Status_Assignment  
(employment_contract_id, contract_status_id, notes)  
    VALUES (p_employment_contract_id,  
p_contract_status_id, p_notes)  
    ON DUPLICATE KEY UPDATE notes = p_notes;  
  
    SELECT 'Status assigned successfully' AS result;  
END//  
DELIMITER ;
```

DELIMITER // и **DELIMITER;** - изменяют и восстанавливают разделитель операторов SQL. Это необходимо, потому что тело хранимой процедуры содержит точки с запятой, которые иначе были бы интерпретированы как конец команды **CREATE PROCEDURE**.

CREATE PROCEDURE AssignContractStatus (...) - создает хранимую процедуру с именем AssignContractStatus.

Данная хранимая процедура содержит несколько входящих параметров:
p_employment_contract_id (айди трудового договора) типа **INT**,
p_contract_status_id (статус договора) типа **INT** и
p_notes(дополнительные примечания) типа **TEXT**.

В операции **INSERT INTO** Contract_Status_Assignment (employment_contract_id, contract_status_id, notes) вставляются значения входящих параметров процедуры **VALUES** (p_employment_contract_id, p_contract_status_id, p_notes).

ON DUPLICATE KEY UPDATE notes = p_notes - обработка ситуации, когда запись с таким ключом уже существует.

END// Завершает тело процедуры. Поскольку в начале мы поменяли разделитель, здесь используется //.

DELIMITER; Возвращает стандартный разделитель команд обратно на точку с запятой ;.

Процедуры к базе данных представлены в приложении Г.

Заключение

В ходе выполнения курсовой работы были разработаны концептуальная, логическая и физическая схемы базы данных, предназначенной для учета личных дел сотрудников в отделе кадров. База данных была реализована с помощью реляционной системы управления базами данных с открытым исходным кодом MySQL. Реализованная база данных позволяет хранить информацию о сотрудниках, их должностях и отделах, о переводах, а также увольнении сотрудников, их отпусках, прохождении обучающих курсов

Была разработана концептуальная модель в нотации Чена, отражающая все бизнес-процессы отдела кадров. Создана логическая схема в нотации IDEF1X, обеспечивающая нормализованную структуру данных. Реализована физическая модель с учетом особенностей СУБД.

База данных приведена к нормальной форме Бойса-Кодда (НФБК), что обеспечивает:

- Отсутствие избыточности данных
- Целостность и непротиворечивость информации
- Эффективное хранение и обработку данных
- Устранение аномалий обновления, удаления и добавления

Разработаны комплексные механизмы для поддержки всех функций отдела кадров:

- Учет приема, перевода и увольнения сотрудников
- Ведение истории должностей и отделов
- Оформление отпусков различных типов
- Управление приказами и трудовыми договорами
- Учет обучения и предыдущего опыта работы

Список использованных источников

1. Гринспан Д., Сегедин Л. *MySQL. Подробное руководство*. — СПб.: Питер, 2021. — 896 с.
2. Корнелиус С., *SQL. Полное руководство* — М.: Вильямс, 2020. — 864 с.
3. Date С. J. *Введение в системы баз данных* / Пер. с англ. — М.: Вильямс, 2019. — 1024 с.
4. Контроль и управление доступом. // *Россия*
URL: <https://bolid.ru/projects/iso-orion/safety-and-security/access-control/?ysclid=majwyuhaon311263410> (дата обращения: 25.02.2025)

Приложение А (обязательное)

Скрипт создания базы данных company_hr.

```
-- Создание базы данных
CREATE DATABASE IF NOT EXISTS company_hr;
USE company_hr;

-- Таблица: Job_position
CREATE TABLE IF NOT EXISTS Job_position (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Таблица: Department
CREATE TABLE IF NOT EXISTS Department (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Таблица: Order_Type
CREATE TABLE IF NOT EXISTS Order_Type (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Таблица: Contract_Status
CREATE TABLE IF NOT EXISTS Contract_Status (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Таблица: Employee
CREATE TABLE IF NOT EXISTS Employee (
    id INT AUTO_INCREMENT PRIMARY KEY,
    last_name VARCHAR(100) NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    middle_name VARCHAR(100),
```



```

    birth_date DATE,
    education_level VARCHAR(255),
    phone_number VARCHAR(20),
    email VARCHAR(255),
    inn VARCHAR(12),
    snils VARCHAR(14),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Таблица: Employment_Contract
CREATE TABLE IF NOT EXISTS Employment_Contract (
    id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT NOT NULL,
    department_id INT NOT NULL,
    position_id INT NOT NULL,
    status_id INT NOT NULL,
    company VARCHAR(255) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE,
    contract_type VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (employee_id) REFERENCES Employee(id)
ON DELETE CASCADE,
    FOREIGN KEY (department_id) REFERENCES
Department(id),
    FOREIGN KEY (position_id) REFERENCES
Job_position(id),
    FOREIGN KEY (status_id) REFERENCES
Contract_Status(id)
);

-- Таблица: Work_Order
CREATE TABLE IF NOT EXISTS Work_Order (
    id INT AUTO_INCREMENT PRIMARY KEY,
    employment_contract_id INT NOT NULL,
    order_date DATE NOT NULL,
    order_type_id INT NOT NULL,
    content TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (employment_contract_id) REFERENCES
Employment_Contract(id) ON DELETE CASCADE,
    FOREIGN KEY (order_type_id) REFERENCES
Order_Type(id)
);

```

```
-- Таблица: Dismissal
CREATE TABLE IF NOT EXISTS Dismissal (
    id INT AUTO_INCREMENT PRIMARY KEY,
    employment_contract_id INT NOT NULL,
    dismissal_date DATE NOT NULL,
    bonus DECIMAL(10,2),
    order_number INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (employment_contract_id) REFERENCES
Employment_Contract(id) ON DELETE CASCADE,
    FOREIGN KEY (order_number) REFERENCES
Work_Order(id)
);
```

```
-- Таблица: Transfer
CREATE TABLE IF NOT EXISTS Transfer (
    id INT AUTO_INCREMENT PRIMARY KEY,
    employment_contract_id INT NOT NULL,
    transfer_date DATE NOT NULL,
    old_position_id INT NOT NULL,
    new_position_id INT NOT NULL,
    old_department_id INT NOT NULL,
    new_department_id INT NOT NULL,
    order_number INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (employment_contract_id) REFERENCES
Employment_Contract(id) ON DELETE CASCADE,
    FOREIGN KEY (old_position_id) REFERENCES
Job_position(id),
    FOREIGN KEY (new_position_id) REFERENCES
Job_position(id),
    FOREIGN KEY (old_department_id) REFERENCES
Department(id),
    FOREIGN KEY (new_department_id) REFERENCES
Department(id),
    FOREIGN KEY (order_number) REFERENCES
Work_Order(id)
);
```

```
-- Таблица: Vacation
CREATE TABLE IF NOT EXISTS Vacation (
    id INT AUTO_INCREMENT PRIMARY KEY,
    employment_contract_id INT NOT NULL,
    order_number INT NOT NULL,
    vacation_type VARCHAR(100) NOT NULL,
```

```

        start_date DATE NOT NULL,
        end_date DATE NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (employment_contract_id) REFERENCES
Employment_Contract(id) ON DELETE CASCADE,
        FOREIGN KEY (order_number) REFERENCES
Work_Order(id)
    );

-- Таблица: Training_Course
CREATE TABLE IF NOT EXISTS Training_Course (
    id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT NOT NULL,
    course_name VARCHAR(255) NOT NULL,
    organization VARCHAR(255) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    certificate VARCHAR(255),
    grade INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (employee_id) REFERENCES Employee(id)
ON DELETE CASCADE
);

```

Приложение Б (обязательное)

Запросы для заполнения базы данных.

-- Добавление должностей

```
INSERT INTO Job_position (name, description) VALUES  
( 'Manager', 'Department head'),  
( 'Developer', 'Software developer'),  
( 'Analyst', 'Business analyst'),  
( 'QA Tester', 'Quality assurance specialist');
```

-- Добавление отделов

```
INSERT INTO Department (name, description) VALUES  
( 'IT', 'Information Technology'),  
( 'HR', 'Human Resources'),  
( 'Finance', 'Finance Department'),  
( 'Marketing', 'Marketing Department');
```

-- Добавление типов приказов

```
INSERT INTO Order_Type (name, description) VALUES  
( 'Employment', 'Employment order'),  
( 'Dismissal', 'Dismissal order'),  
( 'Transfer', 'Transfer order'),  
( 'Vacation', 'Vacation order');
```

-- Добавление типов статусов трудовых договоров

```
INSERT INTO Contract_Status (name, description) VALUES  
( 'Active', 'Active employment contract'),  
( 'Terminated', 'Terminated employment contract'),  
( 'Suspended', 'Suspended employment contract'),  
( 'On Vacation', 'Employee on vacation'),  
( 'Probation', 'Probation period');
```

-- Добавление сотрудников

```
INSERT INTO Employee (last_name, first_name,  
middle_name, birth_date, education_level, phone_number,  
email, inn, snils) VALUES  
( 'Ivanov', 'Ivan', 'Ivanovich', '1990-05-15', 'Higher',  
' +79161234567', 'ivanov@mail.ru', '123456789012', '123-  
456-789-00'),  
( 'Petrov', 'Peter', 'Petrovich', '1985-08-20',  
'Higher', '+79167654321', 'petrov@mail.ru',  
'234567890123', '234-567-890-11');
```

-- Добавление трудовых договоров

```

INSERT INTO Employment_Contract (employee_id,
department_id, job_position_id, company, start_date,
contract_type) VALUES
(1, 1, 2, 'Romashka LLC', '2023-01-15', 'Permanent'),
(2, 1, 1, 'Romashka LLC', '2022-06-01', 'Permanent');

-- Добавление обучающих курсов
INSERT INTO Training_Course (course_name, organization,
description, duration_days) VALUES
('Java Programming', 'IT Academy', 'Advanced Java
programming course', 30),
('Project Management', 'Business School', 'Project
management fundamentals', 20),
('Leadership Skills', 'Management Institute',
'Leadership and team management', 15);

-- Добавление прохождения курсов
INSERT INTO Course_Completion (employee_id,
training_course_id) VALUES
(1, 1),
(1, 2),
(2, 3);

```

Приложение В (обязательное)

Запросы для работы с базой данных.

-- 1.Сотрудники с их текущими статусами договоров

```
SELECT
    e.id AS employee_id,
    CONCAT(e.last_name, ' ', e.first_name) AS
employee_name,
    ec.id AS contract_id,
    ec.start_date,
    ec.contract_type,
    cs.name AS current_status,
    csa.assignment_date AS status_date
FROM Employee e
JOIN Employment_Contract ec ON e.id = ec.employee_id
JOIN Contract_Status_Assignment csa ON ec.id =
csa.employment_contract_id
JOIN Contract_Status cs ON csa.contract_status_id =
cs.id
WHERE ec.end_date IS NULL
ORDER BY e.last_name, cs.name;
```

-- 2. Отпуска сотрудников на текущий год

```
SELECT
    CONCAT(e.last_name, ' ', e.first_name) AS
employee_name,
    v.vacation_type,
    v.start_date,
    v.end_date,
    DATEDIFF(v.end_date, v.start_date) AS
duration_days,
    jp.name AS job_position
FROM Vacation v
JOIN Employment_Contract ec ON v.employment_contract_id
= ec.id
JOIN Employee e ON ec.employee_id = e.id
JOIN Job_position jp ON ec.job_position_id = jp.id
WHERE YEAR(v.start_date) = YEAR(CURDATE())
ORDER BY v.start_date;
```

-- 3. Самые популярные курсы среди сотрудников

```
SELECT
    tc.course_name,
    tc.organization,
```

```

        COUNT(cc.employee_id) AS enrolled_count,
        GROUP_CONCAT(DISTINCT d.name SEPARATOR ', ') AS
departments
FROM Training_Course tc
JOIN Course_Completion cc ON tc.id =
cc.training_course_id
JOIN Employee e ON cc.employee_id = e.id
JOIN Employment_Contract ec ON e.id = ec.employee_id
JOIN Department d ON ec.department_id = d.id
WHERE ec.end_date IS NULL
GROUP BY tc.id, tc.course_name, tc.organization
ORDER BY enrolled_count DESC
LIMIT 10;

```

-- 4. Основная информация о сотрудниках с их
должностями и отделами

```

SELECT
    e.id AS employee_id,
    CONCAT(e.last_name, ' ', e.first_name, ' ',
COALESCE(e.middle_name, '')) AS full_name,
    e.email,
    e.phone_number,
    jp.name AS job_position,
    d.name AS department,
    ec.start_date,
    ec.contract_type
FROM Employee e
JOIN Employment_Contract ec ON e.id = ec.employee_id
JOIN Job_position jp ON ec.job_position_id = jp.id
JOIN Department d ON ec.department_id = d.id
WHERE ec.end_date IS NULL -- только действующие
договоры
ORDER BY e.last_name, e.first_name;

```

-- 5. Сотрудники с их текущими статусами договоров

```

SELECT
    e.id AS employee_id,
    CONCAT(e.last_name, ' ', e.first_name) AS
employee_name,
    ec.id AS contract_id,
    ec.start_date,
    ec.contract_type,
    cs.name AS current_status,
    csa.assignment_date AS status_date
FROM Employee e

```

```
JOIN Employment_Contract ec ON e.id = ec.employee_id
JOIN Contract_Status_Assignment csa ON ec.id =
csa.employment_contract_id
JOIN Contract_Status cs ON csa.contract_status_id =
cs.id
WHERE ec.end_date IS NULL
ORDER BY e.last_name, cs.name;
```


Приложение Г (обязательное)

Запросы для создания триггеров.

```
-- Триггер для автоматического назначения статуса
"Активен" при создании договора
DELIMITER //
CREATE TRIGGER after_employment_contract_insert
AFTER INSERT ON Employment_Contract
FOR EACH ROW
BEGIN
    DECLARE active_status_id INT;

    -- Находим ID статуса "Активен"
    SELECT id INTO active_status_id FROM
Contract_Status WHERE name = 'Active' LIMIT 1;

    -- Если статус найден, назначаем его новому
договору
    IF active_status_id IS NOT NULL THEN
        INSERT INTO Contract_Status_Assignment
        (employment_contract_id, contract_status_id)
        VALUES (NEW.id, active_status_id);
    END IF;
END//
DELIMITER ;

-- Триггер для проверки дат отпуска
DELIMITER //
CREATE TRIGGER before_vacation_insert
BEFORE INSERT ON Vacation
FOR EACH ROW
BEGIN
    IF NEW.start_date > NEW.end_date THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Vacation start date cannot
be after end date';
    END IF;
END//
DELIMITER ;

-- Триггер для логирования изменений статусов договоров
CREATE TABLE IF NOT EXISTS Contract_Status_History (
    id INT AUTO_INCREMENT PRIMARY KEY,
    employment_contract_id INT,
    contract_status_id INT,
```

```

        assignment_date TIMESTAMP DEFAULT
CURRENT_TIMESTAMP,
        changed_by VARCHAR(100),
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

DELIMITER //
CREATE TRIGGER after_contract_status_assignment_insert
AFTER INSERT ON Contract_Status_Assignment
FOR EACH ROW
BEGIN
    INSERT INTO Contract_Status_History
    (employment_contract_id, contract_status_id,
    changed_by)
    VALUES (NEW.employment_contract_id,
NEW.contract_status_id, USER());
END//
DELIMITER ;

```

Запросы для создания процедур.

```
-- Процедура для добавления нового сотрудника с
договором и статусом
DELIMITER //
CREATE PROCEDURE AddNewEmployeeWithContract(
    IN p_last_name VARCHAR(100),
    IN p_first_name VARCHAR(100),
    IN p_middle_name VARCHAR(100),
    IN p_birth_date DATE,
    IN p_education_level VARCHAR(255),
    IN p_phone_number VARCHAR(20),
    IN p_email VARCHAR(255),
    IN p_inn VARCHAR(12),
    IN p_snils VARCHAR(14),
    IN p_department_id INT,
    IN p_job_position_id INT,
    IN p_company VARCHAR(255),
    IN p_start_date DATE,
    IN p_contract_type VARCHAR(100)
)
BEGIN
    DECLARE new_employee_id INT;
    DECLARE new_contract_id INT;

    -- Вставляем данные сотрудника
    INSERT INTO Employee (last_name, first_name,
middle_name, birth_date, education_level, phone_number,
email, inn, snils)
    VALUES (p_last_name, p_first_name, p_middle_name,
p_birth_date, p_education_level, p_phone_number,
p_email, p_inn, p_snils);

    SET new_employee_id = LAST_INSERT_ID();

    -- Вставляем трудовой договор
    INSERT INTO Employment_Contract (employee_id,
department_id, job_position_id, company, start_date,
contract_type)
    VALUES (new_employee_id, p_department_id,
p_job_position_id, p_company, p_start_date,
p_contract_type);

    SET new_contract_id = LAST_INSERT_ID();
```

```

-- Статус "Активен" назначается автоматически через
триггер
    SELECT new_employee_id AS employee_id,
    new_contract_id AS contract_id;
END//
DELIMITER ;

-- Процедура для назначения статуса договору
DELIMITER //
CREATE PROCEDURE AssignContractStatus(
    IN p_employment_contract_id INT,
    IN p_contract_status_id INT,
    IN p_notes TEXT
)
BEGIN
    INSERT INTO Contract_Status_Assignment
    (employment_contract_id, contract_status_id, notes)
    VALUES (p_employment_contract_id,
    p_contract_status_id, p_notes)
    ON DUPLICATE KEY UPDATE notes = p_notes;

    SELECT 'Status assigned successfully' AS result;
END//
DELIMITER ;

-- Процедура для получения текущих статусов договора
DELIMITER //
CREATE PROCEDURE GetContractStatuses(IN
p_employment_contract_id INT)
BEGIN
    SELECT
        cs.name AS status_name,
        cs.description AS status_description,
        csa.assignment_date,
        csa.notes
    FROM Contract_Status_Assignment csa
    JOIN Contract_Status cs ON csa.contract_status_id =
cs.id
    WHERE csa.employment_contract_id =
p_employment_contract_id
    ORDER BY csa.assignment_date DESC;
END//
DELIMITER ;

```

```

-- Процедура для записи сотрудника на курс (только
связь)
DELIMITER //
CREATE PROCEDURE EnrollEmployeeInCourse(
    IN p_employee_id INT,
    IN p_training_course_id INT
)
BEGIN
    INSERT INTO Course_Completion (employee_id,
training_course_id)
    VALUES (p_employee_id, p_training_course_id)
    ON DUPLICATE KEY UPDATE training_course_id =
training_course_id;

    SELECT 'Employee enrolled in course successfully'
AS result;
END//
DELIMITER ;

-- Процедура для получения курсов сотрудника
DELIMITER //
CREATE PROCEDURE GetEmployeeCourses(IN p_employee_id
INT)
BEGIN
    SELECT
        tc.id AS course_id,
        tc.course_name,
        tc.organization,
        tc.description,
        tc.duration_days,
        cc.created_at AS enrollment_date
    FROM Course_Completion cc
    JOIN Training_Course tc ON cc.training_course_id =
tc.id
    WHERE cc.employee_id = p_employee_id
    ORDER BY cc.created_at DESC;
END//
DELIMITER ;

-- Процедура для получения сотрудников на курсе
DELIMITER //
CREATE PROCEDURE GetCourseParticipants(IN
p_training_course_id INT)
BEGIN
    SELECT

```

```

        e.id AS employee_id,
        e.last_name,
        e.first_name,
        e.middle_name,
        e.email,
        jp.name AS job_position,
        d.name AS department,
        cc.created_at AS enrollment_date
FROM Course_Completion cc
JOIN Employee e ON cc.employee_id = e.id
JOIN Employment_Contract ec ON e.id =
ec.employee_id
JOIN Job_position jp ON ec.job_position_id = jp.id
JOIN Department d ON ec.department_id = d.id
WHERE cc.training_course_id = p_training_course_id
ORDER BY e.last_name, e.first_name;
END//
DELIMITER ;

```